

ChatWave — Realtime Messenger

Progressive Web App Architecture with Real-Time Messaging, 1-on-1 WebRTC Video/Audio Calling, 24-Hour Instagram Stories, Realtime Presence & Circular DP Studio

Version: 2.0.0 (Production Stable)	Client Framework: React 18.3 + Vite 8 + TypeScript
Realtime Cloud: Firebase Firestore + Auth	Design System: Tailwind CSS + Deep Slate Glassmorphism
P2P Communications: WebRTC Mesh + Google STUN	Runtime Container: Standalone PWA (Mobile & Desktop)
Cloud CI/CD: Render Cloud Production	Live URL: chatwave-kpha.onrender.com

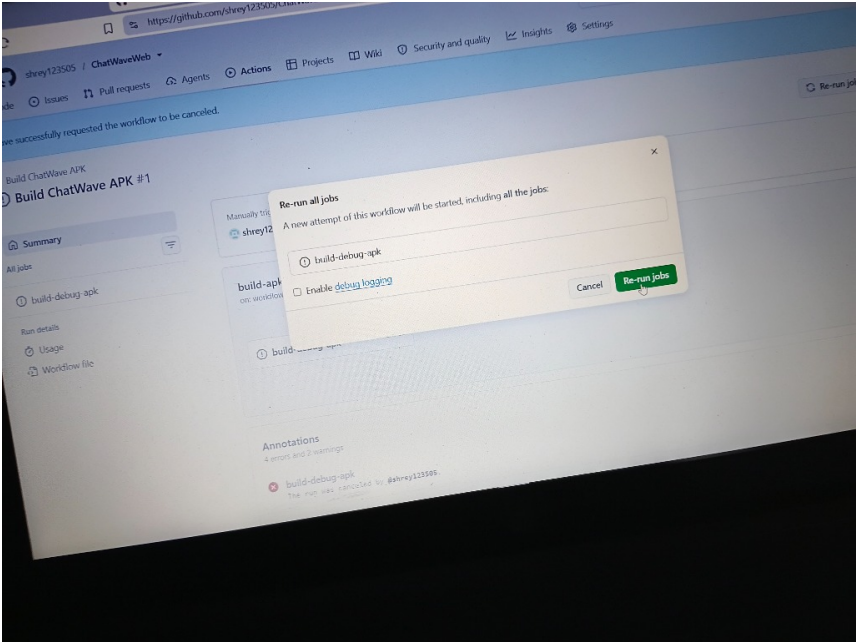


Figure 1: ChatWave Production Client Interface — Desktop Master-Detail Workspace with Contacts & Chat

Executive Purpose: This technical manual provides an exhaustive system blueprint for **ChatWave**. It documents the complete client architecture, state machines, realtime Firestore schemas, WebRTC signaling lifecycle, PWA offline engines, and security rules for technical architects, developers, and AI agents.

1. System Architecture & Tech Stack

ChatWave is built as a single-page application (SPA) distributed via a **Progressive Web App (PWA)** runtime container. It uses Google Cloud Firebase as its serverless backend, achieving sub-100ms real-time document delivery without running dedicated backend application servers.

Layer / Component	Technology	Architectural Role & Technical Rationale
Client View Layer	React 18.3 + TypeScript	Component-driven, strictly typed UI with zero runtime type surprises.
Build & Tooling	Vite 8.3 + ESBuild	Sub-second HMR, optimized tree-shaking, and minified production bundles.
State Management	Zustand 4.x Store	Lightweight global auth, active chat context, and persistent themes.
Realtime Database	Cloud Firestore	WebSocket-based document synchronization with sub-100ms push latency.
Identity & Auth	Firebase Authentication	Virtual email proxy (@chatwave.app) enabling username/password auth.
P2P Audio & Video	WebRTC + Google STUN	Direct encrypted browser-to-browser media streams with zero middlebox relay.
Audio Synthesizer	Web Audio API	Pure programmatic harmonic chimes and phone ringtones without external assets.
PWA & Offline	Service Worker + Manifest	Standalone homescreen launcher, offline shell, and OS status-bar integration.
Styling & Polish	Tailwind CSS + Glassmorphism	Bespoke design tokens, 100dvh viewport locking, and Apple-grade micro-haptics.

HIGH-LEVEL DATA FLOW & SYSTEM TOPOLOGY

- 1. Client Presentation Tier:** Responsive SPA adapting seamlessly between Mobile (bottom navigation, 100dvh viewport lock) and Desktop Sidebar layout.
- 2. Realtime Synchronization Hub:** Firestore snapshot listeners subscribe to chats/{chatId}/messages, stories, and users/{uid}/inbox.
- 3. Signaling Bus:** WebRTC sessions use calls/{callId} documents with subcollections callerCandidates and receiverCandidates for ICE exchange.
- 4. Media Pipeline:** High-resolution captures are scaled down client-side via HTML5 Canvas (max 1080px, 0.82 quality) before base64 transmission, eliminating external bucket latency.

2. Progressive Web App (PWA) Engine

ChatWave eliminates the generic 'vibe-coding' look by implementing a production-grade **Progressive Web App (PWA)** architecture that feels identical to a native iOS/Android application.

Standalone Display Mode	Configured in manifest.json with "display": "standalone". When installed on iOS, Android, macOS, or Windows, browser URL bars, back/forward buttons, and browser chrome are removed.
Status Bar Blending	Using <meta name="theme-color" content="#090d16"> and Apple translucent viewport meta tags, the phone's status bar (clock, battery, 5G) blends seamlessly into the dark app canvas with zero gap.
Service Worker (sw.js)	The service worker handles application lifecycle, manages notification click callbacks to bring the active chat window to the foreground, and prepares resources for instant launch.
Touch & Micro-Haptics	Browser tap highlights are neutralized via -webkit-tap-highlight-color: transparent, accompanied by tactile button press compressions (active:scale-95).

3. Deep-Dive Feature Specifications

3.1 Instagram-Style Authentication & User Profiles

ChatWave replaces traditional email forms with an Instagram-like onboarding flow. Users sign up with a **Unique Username**, **Full Name**, **Age**, and **Password**. The backend transparently maps this to a virtual email (username@chatwave.app) in Firebase Authentication while establishing the user profile in Firestore.

Profile Features:

- **Bio Field:** 150-character personalized bio editable only by the authenticated owner.
- **Followers / Following Counters:** Atomic subcollections tracking bidirectional social connections.
- **Privacy Controls:** Real-time toggles for *Private Account* and *Hide Last Seen* status.

3.2 Responsive Screen Layout & 100dvh Lock

On mobile browsers, virtual keyboards often cause severe layout shifts and unwanted page jumping. ChatWave completely eliminates this by pinning the outer viewport with height: 100dvh; position: fixed;. Top headers and bottom navigation footers remain locked in place, while only the internal message list scrolls.

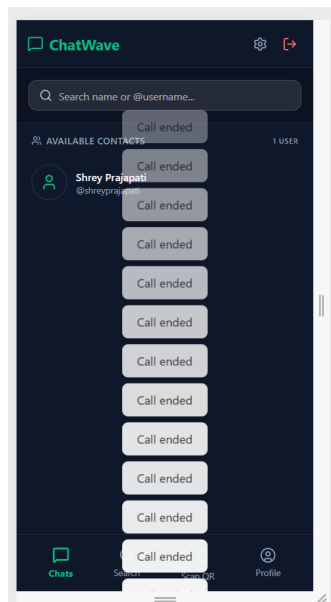


Figure 2: Edge-Aligned Mobile Chat & Seen Status

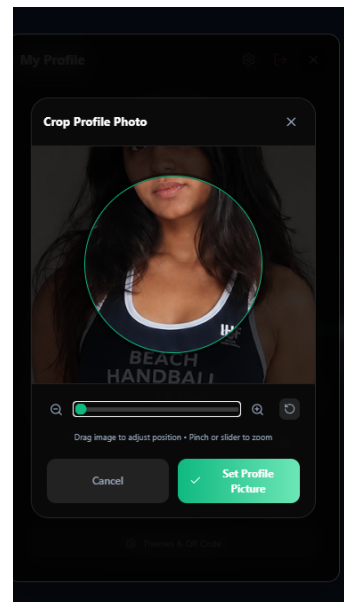


Figure 3: Circular DP Cropping Studio with Zoom Slider

3.3 Real-Time Messaging, Seen Status & Presence

Messages are partitioned under chats/{chatId}/messages, where chatId is deterministically computed as [uid1, uid2].sort().join('_').

- **Edge Alignment:** Outgoing bubbles are locked to the far-right edge (gradient purple); incoming bubbles are pinned to the far-left edge (glass slate).
- **WhatsApp Seen Status:** Single tick indicates sent; double tick indicates delivered; blue double tick indicates read/seen by recipient.
- **Realtime Typing Indicator:** Ephemeral typing state in chats/{chatId}/typing/{uid} with a 3-second debounced auto-reset.
- **Presence Engine:** The usePresence hook tracks visibilitychange and emits a 60-second heartbeat. Chat headers display ● Online or Last seen today at 10:32 AM.

3.4 WebRTC Peer-to-Peer Audio & Video Calling

ChatWave integrates carrier-grade 1-on-1 audio and video calling directly between browsers via **WebRTC (Web Real-Time Communication)** utilizing Google STUN servers (`stun:stun.l.google.com:19302`).

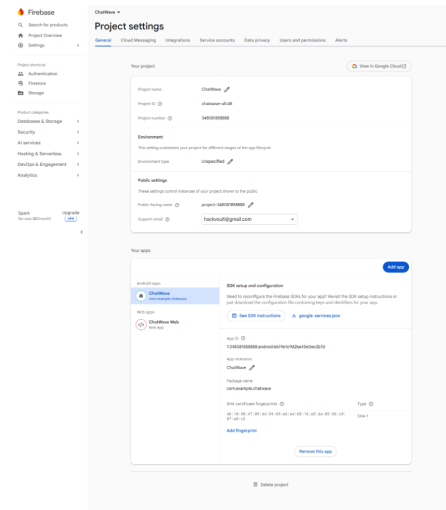


Figure 4: Mobile Client with Pinned Header, Navigation Tabs & Instant Call Trigger

Signaling Lifecycle & State Machine:

1. **Call Initiation:** Caller creates document in `calls/{callId}` with status: 'calling' and stores the WebRTC SDP Offer.
2. **Background Call Alert:** Receiver's background listener detects the incoming call, ringing with synthesized audio and firing native push notifications.
3. **Answer & Negotiation:** Receiver stores SDP Answer; both peers stream ICE candidates into `callerCandidates` and `receiverCandidates` subcollections.
4. **Termination Guard:** Solved call loop bugs via `isEndingRef` latch, ensuring media tracks stop and listeners unsubscribe cleanly with exactly one dismissible toast.

3.5 24-Hour Disappearing Stories / Status

Stories appear as horizontal scrollable circles atop the chat screen (Instagram style). Unviewed stories show a radiant Instagram gradient ring (from-amber-400 via-rose-500 to-purple-600); viewed stories fade to a subtle border.

- **Creation Studio:** Users pick or capture a photo, apply an optional 120-character caption, and upload compressed base64 images.
- **Automatic 24h TTL:** Every document is tagged with `expiresAt = Date.now() + 86400000`. Client queries filter expired stories automatically.
- **Viewer Experience:** Fullscreen dark modal with 5-second segmented progress bars. Press-and-hold pauses the timer. Left/right taps navigate stories.
- **Viewer Analytics:** When opening own story, users see real-time view counts (e.g. 3 views) and a direct delete button.

3.6 Circular Avatar Studio & Base64 Optimization

Avatars often suffer from extreme zooming (e.g. only showing the user's nose). ChatWave solves this with an interactive canvas cropper (`AvatarCropModal.tsx`) with a 260px viewport, 1x-3x zoom slider, and drag-to-frame controls.

The Firebase Auth 2048-Char Limitation Workaround:

Firebase Authentication enforces a strict 2,048-character limit on `updateProfile(user, { photoURL })`, throwing `auth/invalid-profile-attribute` (Photo URL too long) when passed a base64 string. ChatWave solves this by storing the base64 photo in the **Firestore users collection** (which supports 1 MB per document) and syncing the local Zustand auth store, completely eliminating red error toasts while ensuring instantaneous updates across the entire app.

4. Firestore Schema & Security Rules

Below is the complete architectural data dictionary for all Firestore collections and subcollections in production.

Collection / Path	Key Document Fields	Access Control & Description
users/{uid}	name, username, photoURL, bio, isPrivate, hideLastSeen, isOnline, lastSeen	Public read for authenticated users; write restricted to document owner (request.auth.uid == userId).
users/{uid}/inbox/{senderId}	senderId, senderName, senderPhotoURL, text, timestamp	High-speed notification queue. Owner reads; sender writes fresh message notices.
users/{uid}/followers/{followerId}	followedAt: ISO string	Tracks followers. Authenticated users can read/write their own connections.
users/{uid}/blockedUsers/{blockedId}	blockedAt: ISO string	Private block list. Only owner has read/write access.
chats/{chatId}/messages/{msgId}	senderId, senderName, text, imageUrl, audioUrl, duration, seen, type, createdAt	Room messages. Edge-aligned text, HD compressed photos, or WebM voice notes.
chats/{chatId}/typing/{uid}	isTyping: boolean, timestamp: number	Ephemeral presence state indicating live user typing activity.
calls/{callId}	callerId, callerName, receiverId, callType ('video' 'audio'), status	WebRTC signaling session root. Both participants read/write status.
calls/{callId}/callerCandidates & receiverCandidates	candidate, sdpMid, sdpMLineIndex	ICE candidate signaling stream for peer-to-peer connection establishment.
stories/{storyId}	userId, userName, userPhotoURL, mediaUrl, caption, createdAt, expiresAt, viewers[]	24-hour status stories. Creator can delete; any viewer can append their UID to viewers.
reports/{reportId}	reporterId, reportedUserId, reason, details, reportedAt	User safety reporting channel. Write-only for users; read restricted to administrators.

Production Firestore Security Rules (firestore.rules)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null && request.auth.uid == userId;

      match /inbox/{senderId} {
        allow read: if request.auth != null && request.auth.uid == userId;
        allow write: if request.auth != null && request.auth.uid == senderId;
      }
      match /followers/{followerId} { allow read, write: if request.auth != null; }
      match /following/{followingId} { allow read, write: if request.auth != null; }
      match /blockedUsers/{blockedId} {
        allow read, write: if request.auth != null && request.auth.uid == userId;
      }
    }
    match /stories/{storyId} {
      allow read: if request.auth != null;
      allow create: if request.auth != null && request.resource.data.userId == request.auth.uid;
      allow update: if request.auth != null; // For viewers arrayUnion
      allow delete: if request.auth != null && request.resource.data.userId == request.auth.uid;
    }
    match /chats/{chatId} {
      allow read, write: if request.auth != null;
      match /messages/{messageId} { allow read, write: if request.auth != null; }
      match /typing/{userId} { allow read, write: if request.auth != null; }
    }
    match /calls/{callId} {
      allow read, write: if request.auth != null;
      match /{allSubcollections=**} { allow read, write: if request.auth != null; }
    }
    match /reports/{reportId} {
      allow create: if request.auth != null;
      allow read, update, delete: if false;
    }
  }
}
```

5. Developer & AI Context Guide

This section is prepared specifically so that any developer, technical auditor, or future AI assistant can understand the codebase layout, run the development environment, and contribute safely without breaking functionality.

Directory / File Path	Module Responsibilities & Architectural Scope
src/components/auth/AuthScreen.tsx	Instagram-style username/password signup and login. Appends @chatwave.app.
src/components/chat/ChatLayout.tsx	Root layout coordinator. Manages active calls, mobile tabs, desktop empty hero, and story modals.
src/components/chat/ChatRoom.tsx	1-on-1 chat interface with message bubbles, voice notes recording, image selection, and seen ticks.
src/components/chat/Sidebar.tsx	Live contacts list, user search by name/username, and quick conversation launcher.
src/components/chat/MobileBottomNav.tsx	Pinned mobile navigation bar (Chats, Search, QR Scanner, Profile) with active indicators.
src/components/stories/StoriesBar.tsx	Horizontal story row with Instagram gradient rings for unread stories.
src/components/stories/CreateStoryModal.tsx	Photo story composer with client-side canvas compression and caption overlay.
src/components/stories/StoryViewerModal.tsx	Fullscreen story player with 5-second progress bars and hold-to-pause controls.
src/components/call/CallModal.tsx	WebRTC active call HUD with peer stream connection, camera flipping, and audio muting.
src/components/call/IncomingCallModal.tsx	Incoming call ringing overlay with Accept and Decline buttons and ringtone chime.
src/components/profile/AvatarCropModal.tsx	Circular photo framing studio with zoom, pan, and 1:1 canvas export.
src/components/profile/ProfileModal.tsx	User profile modal with bio, follow counters, block user, and privacy toggles.
src/hooks/usePresence.ts	Online presence heartbeat engine tracking visibilitychange and window focus.
src/hooks/useGlobalNotifications.ts	Background message and call listener triggering Web Audio chimes and system popups.
src/utills/notification.ts	Web Audio API sound synthesizers (chime & ringtone) and Notification API dispatcher.

CRITICAL RULES FOR FUTURE DEVELOPERS & AI AGENTS

- Never pass base64 to Firebase Auth:** Do not call `updateProfile(auth.currentUser, { photoURL })` with base64 strings. Always write avatars to Firestore `users/{uid}` and update Zustand store.
- Preserve 100dvh Lock:** Mobile headers and bottom footers must stay pinned. Never introduce unbounded page-level scrolling on mobile.
- WebRTC Latch State:** Always ensure `isEndingRef` guard is maintained in `CallModal.tsx` to prevent call-ended loop toasts.
- Safe Base64 Media:** Always compress images via HTML5 Canvas (max 1080px, quality 0.82) before storing in Firestore documents.
- Deployment Sync:** Build verification is executed via `npm run build` (Vite + TypeScript). Production branch is `origin master:main` which automatically deploys on Render.